Practical 5

Configure a Basic Load Balancer

Objective: Simulate load balancing in a lab setup using HAProxy to distribute requests between two web servers.

Steps to Implement

Step 1: Set up the Environment

1. **Prepare a Linux Machine:**
   Ensure you have a Linux system (e.g., Ubuntu, CentOS).
   - **Update system packages:**

Code:

```
sudo apt update && sudo apt upgrade -y          # For Ubuntu
sudo yum update -y                              # For CentOS
```

```
vboxuser@ubuntu1:~$ sudo apt update && sudo apt upgrade -y
[sudo] password for vboxuser:
Get:1 file:/cdrom noble InRelease
Ign:1 file:/cdrom noble InRelease
Get:2 file:/cdrom noble Release
Err:2 file:/cdrom noble Release
  File not found - /cdrom/dists/noble/Release (2: No such file or directory)
Hit:3 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:4 http://in.archive.ubuntu.com/ubuntu noble InRelease
Get:5 http://in.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Hit:6 http://in.archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
E: The repository 'file:/cdrom noble Release' no longer has a Release file.
N: Updating from such a repository can't be done securely, and is therefore disa
bled by default.
N: See apt-secure(8) manpage for repository creation and user configuration deta
ils.
vboxuser@ubuntu1:~$
```

2. **Install HAProxy:**

Code:

```
sudo apt install haproxy -y          # For Ubuntu
sudo yum install haproxy -y          # For CentOS
```

```
vboxuser@ubuntu1:~$ sudo apt install haproxy -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Suggested packages:
  vim-haproxy haproxy-doc
The following NEW packages will be installed:
  haproxy
0 upgraded, 1 newly installed, 0 to remove and 3 not upgraded.
Need to get 2,038 kB of archives.
After this operation, 4,631 kB of additional disk space will be used.
Get:1 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 haproxy amd64 2.8.5-1ubuntu3.1 [2,038 kB]
Fetched 2,038 kB in 4s (531 kB/s)
Selecting previously unselected package haproxy.
(Reading database ... 152091 files and directories currently installed.)
Preparing to unpack .../haproxy_2.8.5-1ubuntu3.1_amd64.deb ...
Unpacking haproxy (2.8.5-1ubuntu3.1) ...
Setting up haproxy (2.8.5-1ubuntu3.1) ...
invoke-rc.d: policy-rc.d denied execution of start.
Created symlink /etc/systemd/system/multi-user.target.wants/haproxy.service → /usr/lib/systemd/system/haproxy.service.
/usr/sbin/policy-rc.d returned 101, not running 'start haproxy.service'
Processing triggers for rsyslog (8.2312.0-3ubuntu9) ...
invoke-rc.d: policy-rc.d denied execution of try-restart.
Processing triggers for man-db (2.12.0-4build2) ...
vboxuser@ubuntu1:~$
```

Step 2: Set up Web Servers

1. Install Python (if not already installed):

Code:

sudo apt install python3 -y # For Ubuntu

sudo yum install python3 -y # For CentOS

```
vboxuser@ubuntu1:~$ sudo apt install python3 -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
python3 is already the newest version (3.12.3-0ubuntu2).
python3 set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 3 not upgraded.
vboxuser@ubuntu1:~$
```

2. Create Two Simple HTTP Servers:

- **Server 1:**
  Open a terminal and run:

Code:

mkdir ~/server1 && cd ~/server1
echo "Hello from Server 1" > index.html
python3 -m http.server 8081

```
vboxuser@ubuntu1:~$ mkdir ~/server1 && cd ~/server1
vboxuser@ubuntu1:~/server1$ echo "Hello from Server 1" > index.html
vboxuser@ubuntu1:~/server1$ python3 -m http.server 8081
Serving HTTP on 0.0.0.0 port 8081 (http://0.0.0.0:8081/) ...
```

- **Server 2:**
  Open another terminal and run:

Code:

```
mkdir ~/server2 && cd ~/server2
echo "Hello from Server 2" > index.html
python3 -m http.server 8082
```

Step 3: Configure HAProxy

1. **Edit the HAProxy Configuration File:**
   Open the configuration file:

Code:

```
sudo nano /etc/haproxy/haproxy.cfg
```

Add the following configuration:

Code:

```
global
    log /dev/log local0
    log /dev/log local1 notice
    daemon

defaults
    log global
    option httplog
    timeout connect 5000ms
    timeout client  50000ms
    timeout server  50000ms

frontend http_front
    bind *:80
    default_backend http_back

backend http_back
    balance roundrobin
    server server1 127.0.0.1:8081 check
    server server2 127.0.0.1:8082 check
```

2. **Restart HAProxy Service:**
   Save the file and restart HAProxy:

Code:

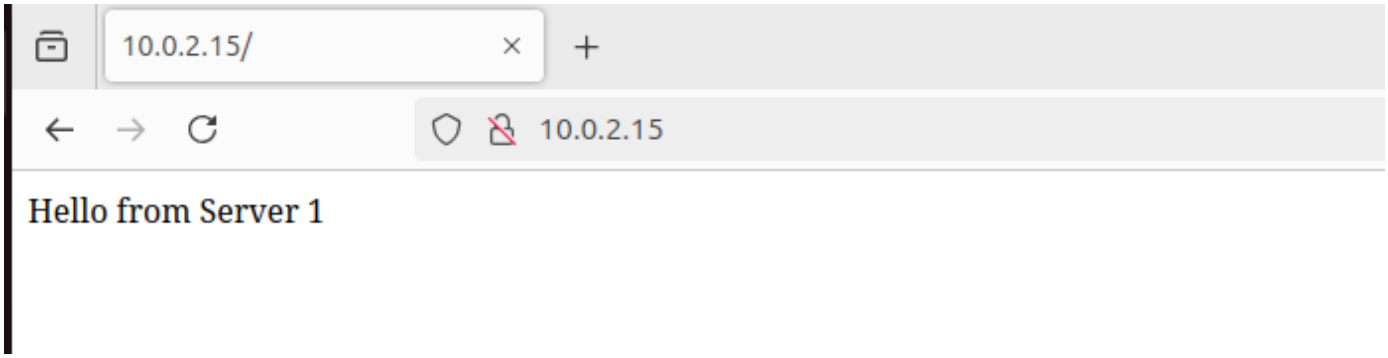
sudo systemctl restart haproxy

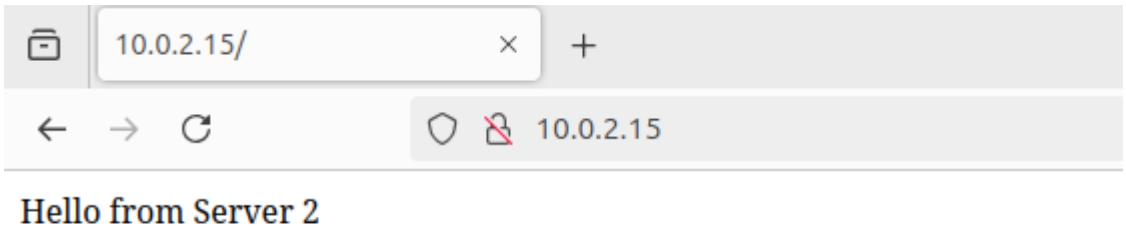
sudo systemctl enable haproxy

```
vboxuser@Ubuntu:~$ sudo systemctl restart haproxy
[sudo] password for vboxuser:
vboxuser@Ubuntu:~$ sudo systemctl enable haproxy
Synchronizing state of haproxy.service with SysV service script with /usr/lib/sy
stemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable haproxy
vboxuser@Ubuntu:~$
```

Step 4: Test the Load Balancer

1. **Open a web browser and navigate to http://<Linux-machine-IP>.**
2. **Refresh the page multiple times to observe requests being distributed between the two servers.**
   - **You should alternately see "Hello from Server 1" and "Hello from Server 2."**





Hello from Server 2

Outcome

You have successfully configured a basic load balancer using HAProxy. This practical exercise demonstrates how load balancing can distribute traffic evenly across multiple servers, ensuring better resource utilization and availability.